Acxelin TLS: How It Works

Oct 29, 2026 · @QubitMan

In one page

Acxelin TLS puts a quantum-safe lock on the connections a company already has, and the company's software doesn't change.

Whenever two computers talk over a network, they first agree on a secret key and then scramble everything they say with it. A phone app and a bank, two offices, or a website and its database all work this way. The agreement is called TLS. Today's TLS relies on maths that a large quantum computer will be able to break.

The danger has already started. An attacker can record scrambled traffic today and unscramble it years from now, once a quantum computer exists. This is called **harvest now, decrypt later**. Bank records, health data and contracts must stay secret for longer than that.

Acxelin TLS has three parts:

Part	Think of it as	What it does
The edge	A guard at the front door	Stands in front of any service (a website, a database, a message system). Lets in only visitors who use the quantum-safe lock, then walks them through to the service.
The certificate authority (CA)	The passport office	Hands out the ID cards (certificates) that prove a server or a client is who it says. The cards carry quantum-safe signatures.
The console	The front desk	A web page on your own computer where you set up the CA, issue and cancel ID cards, and check on the guards, without typing commands.

The software behind the edge stays as it is. It keeps talking plain traffic on its own machine, and the edge does the quantum-safe part.

The quantum-safe maths comes from the US standards body NIST, which approved it in August 2024. **ML-KEM** is used to agree the secret key, and **ML-DSA** is used to sign the ID cards.

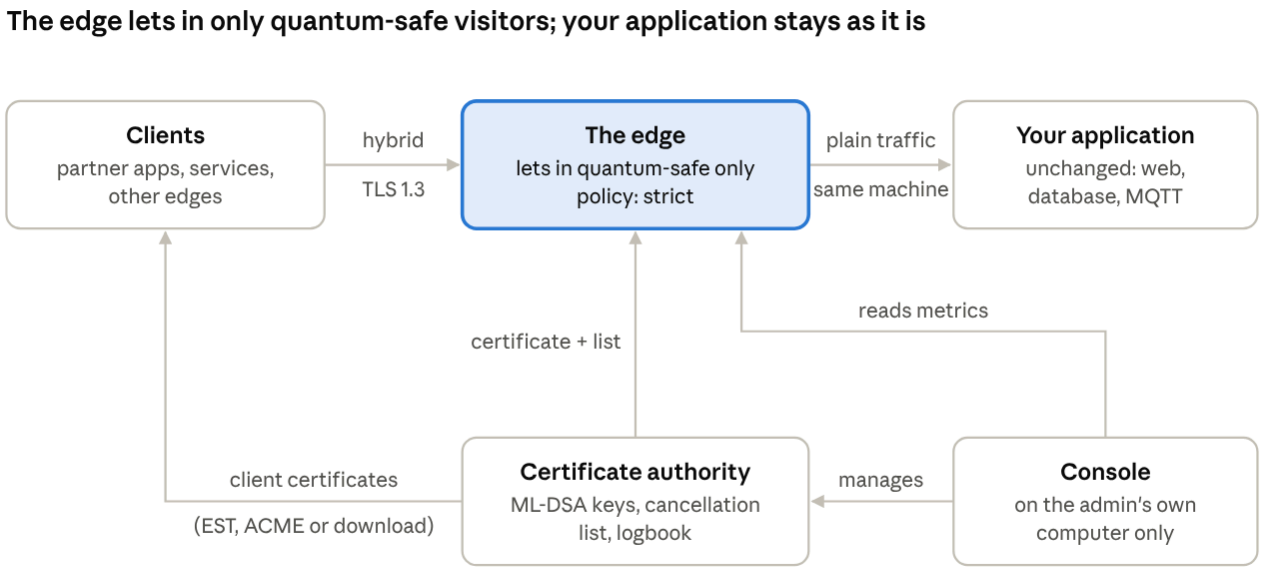
The ideas you need

Six ideas explain the whole product. Each starts with an everyday picture, then gives the technical name.

1. **TLS is a sealed envelope.** When two computers talk, TLS puts every message in an envelope only the other side can open. First they exchange a short greeting, the *handshake*, to agree on a secret key. Everything after that is scrambled with the key (AES-256-GCM or ChaCha20-Poly1305). TLS 1.3 is the current version.
2. **Key exchange is agreeing a secret in public.** Two people agree on a secret while everyone listens, and never say the secret out loud. Today this uses elliptic-curve maths (X25519). A quantum computer could solve that maths, so recorded handshakes could be opened later.
3. **ML-KEM is the quantum-safe way to agree a secret.** It uses a different kind of maths (lattices) that no known quantum method breaks. NIST published it as FIPS 203 in August 2024.
4. **Hybrid means two locks on one door.** Acxelin TLS uses **X25519MLKEM768**, the old lock and the quantum-safe lock together, so an attacker has to break both. If ML-KEM ever turned out weak, X25519 still protects you today. If a quantum computer breaks X25519, ML-KEM still protects you. Chrome, Edge, Firefox, Cloudflare and Google already use the same pair.
5. **Certificates are ID cards for computers.** A certificate says "this is web.example.com, and here is its public key", signed by someone you trust. It stops an attacker from pretending to be your server. **ML-DSA** (FIPS 204) is the quantum-safe signature on those cards.
6. **A certificate authority (CA) is the passport office.** The CA is whoever signs the ID cards. Public websites buy theirs from commercial CAs. Inside a company (servers talking to servers, partner connections, devices), the company can run its **own** CA, and Acxelin TLS gives it one that signs with ML-DSA.

Mutual TLS (mTLS) means both sides show an ID card, not just the server. A bank's partner connection uses it so that only approved partners can connect at all.

At a glance



Acxelin TLS · 5 parts and what passes between them

Clients reach the edge over quantum-safe TLS 1.3, and the edge passes plain traffic to the unchanged application on the same machine. The CA gives the edge and the clients their ML-DSA certificates. The console, on the administrator's own computer, runs the CA and reads each edge's metrics.

Tech stack

Acxelin TLS is written in Python and does its cryptography through OpenSSL 3.5, with one Python library (`cryptography`) besides. There is no C code of our own, and it never runs the `openssl` command.

Piece	What we use	Why
Language	Python 3.11 to 3.14	Runs on Linux, Windows and macOS, and is easy to read and check
The handshake	OpenSSL 3.5's TLS library, called directly from Python (`acxelin_core/openssl.py` , 521 lines)	Python's built-in `ssl` module can't choose quantum-safe key exchanges or say which one was used. OpenSSL 3.5 is the first version with ML-KEM and ML-DSA built in.

Piece	What we use	Why
ID cards and keys	`cryptography` 49 or newer	Makes certificates, cancellation lists and requests, and creates ML-DSA keys
Optional	`boto3`, only to keep the CA's key in AWS KMS	Installed only if you ask for the `kms` extra
Console	One HTML page served by Python's own web server on 127.0.0.1	Nothing extra to install, and no outside scripts
Settings	TOML files, read by Python's standard library	Settings a person can read and edit
Packaging	An offline folder of Python packages (a "wheelhouse") with `install.sh` and `install.ps1`, plus a Docker image	Installs with no internet, in one command
Tests	`pytest` (118 test functions in the TLS product, more in core), `hypothesis`, Playwright and axe-core for the console in a real browser, and real nginx, Redis, PostgreSQL, MQTT and `certbot`	Proves it works with other people's software, not only with itself
Automated checks	GitHub Actions on Ubuntu, Windows and macOS with Python 3.11 to 3.14, a Debian 13 container with OpenSSL 3.5, a Docker image scan (Trivy), CodeQL and a speed job	Every change is checked on every platform before a release

The code is split into two packages

Package	Lines of Python	Holds
`acxelin-core` (version 1.1.0)	about 2,800	The shared machinery: the bridge to OpenSSL, the TLS server, the certificate authority, machine enrolment (EST) and the console's safety rules. Every Acxelin product uses it.
`acxelin-tls` (version 1.0.2)	about 2,300, plus the console page	The product itself: the edge, its policies, the commands, ACME and the console's pages.

The rule between them is simple. **Core knows *how* to do things; the product decides *what* is allowed.** Core can run a TLS handshake; the TLS product decides which algorithms that handshake may use.

The certificate authority

The CA is a folder plus one secret key, and everything it signs uses ML-DSA.

What's in the CA folder

File	What it is	Think of it as
`ca.key`	The CA's secret key, locked with a passphrase if you set one	The passport office's official stamp. Whoever holds it can make IDs.
`ca.crt`	The CA's own certificate (the "root")	A public sample of the stamp that everyone compares against. You give this file to clients so they trust you.
`crl.pem`	The cancellation list: the serial numbers of cancelled certificates, signed and valid for 7 days	The list of stolen passports at the border
records file	One line per certificate issued: serial, name, kind, algorithm, expiry, status	The office's register
`audit.jsonl`	Who did what and when; only the owner can read it	The office's logbook

Its life, step by step

1. **Create** (`ca init` , or Create in the console). This makes a root CA that signs with ML-DSA-87, the strongest level, valid for 10 years. It can also be an SLH-DSA root (a very cautious, hash-based signature) or a middle CA signed by another root, so the root can stay switched off in a safe.
2. **Issue** (`ca issue`). This makes a new ML-DSA-65 key and certificate for a name. There are three kinds: `server` proves a server, `client` proves a client (for mutual TLS), and `site` does both (used by the VPN). A certificate lasts 397 days by default and never longer than the CA itself. Extra names and IP addresses can go in too.
3. **Sign a request** (`ca sign-csr`). This is for a key made on another machine, so the secret key never has to travel.

4. **Cancel** (`ca revoke`). This adds the serial number to the cancellation list, with a reason, and signs the list again. Edges refuse that certificate at their next check.
5. **Look after it** (`ca maintain`, run once a day). This renews certificates that run out within 30 days and signs the cancellation list again. If the list itself runs out, edges using mutual TLS refuse everyone. That is on purpose: it is safer to stop than to let a cancelled partner in.

Where the CA's key can live

Where	When to use it
A key file (the default)	Small teams; the file is locked with the passphrase
AWS KMS	The key never leaves Amazon's hardware
Any hardware security module with a signing command	A bank's existing key hardware

The KMS and hardware options are built and have automated tests, but they have not yet been tried against a real hardware module or cloud account.

Getting certificates to many machines

- **EST** (`ca serve`, `ca token`, `ca enroll`). The operator makes a one-time code for one name. The machine creates its own key and signs itself up over a quantum-safe connection. Only a fingerprint of each code is stored. Renewal happens over mutual TLS and cannot change the machine's name.
- **ACME** (`ca acme`). This is the same method Let's Encrypt uses, so standard tools such as `certbot` can get and renew ML-DSA certificates (with `--csr`).
- **Publish** (`ca publish`). This serves the cancellation list and `ca.crt` over the web, so edges can pick up cancellations by themselves (`crl_url`, checked every 60 seconds by default).

How a connection is decided

A visitor gets through only if it uses a quantum-safe key, trusts an ML-DSA ID card and, with mutual TLS, shows a valid ID card of its own. Here is one connection, step by step.

1. **The visitor knocks.** It opens a connection to the edge, for example on port 8443. The edge first checks how busy it is: at most 512 open connections per route, and at most 50 new ones a second from one address. Over the limit, the visitor is turned away

before any cryptography runs.

2. **The visitor says hello.** Its first message lists the key exchanges it can do. Under the `strict` policy, the edge accepts only the quantum-safe ones: `X25519MLKEM768`, `SecP384r1MLKEM1024` or `SecP256r1MLKEM768`. A visitor that offers only the old kind is refused, and the log says why: *the client offers none of the key exchanges this policy allows*.
3. **They agree a key.** Both sides work out the same secret using both locks at once. A recording of this step is useless to a future quantum computer.
4. **The edge shows its ID card.** It sends its ML-DSA certificate, and the visitor checks it against `ca.crt`.
5. **The visitor shows its ID card (mutual TLS only).** The edge checks that the card was signed by the CA and is not on the cancellation list. Missing, unknown or cancelled: refused.
6. **Traffic flows.** The edge opens a plain connection to the real service (for example 127.0.0.1:8080) and passes the data both ways. The service never knows TLS was involved.
7. **Safety nets.** A handshake must finish within 10 seconds. A connection that sits idle for an hour is closed. New certificates on disk are picked up without a restart. Every key exchange used is counted for the metrics page.

The three policies

Policy	Key exchange allowed	ID cards allowed	Meant for
`strict` (the default)	Quantum-safe hybrid only	ML-DSA only	Your own services and partners
`transition`	Quantum-safe first; the old kinds (X25519, P-256, P-384) still accepted	Any, with an optional ordinary backup certificate	The move-over period, and public websites
`cnsa2`	ML-KEM-1024 only (`SecP384r1MLKEM1024`, `MLKEM1024`)	ML-DSA-87 only, with AES-256 only	US government "CNSA 2.0" rules

The scrambling after the handshake is the same in all three: AES-256-GCM, ChaCha20-Poly1305 or AES-128-GCM. Only TLS 1.3 is accepted; older versions never are.

Two directions

- **terminate**: quantum-safe visitors → the edge → the plain service. This is the usual case.

- **originate**: an old program on this machine → the edge → a quantum-safe connection to an edge somewhere else. An old program that can't be changed can still talk safely across the network, with one edge at each end.

Seeing it in Wireshark

A recording of the network shows the edge using quantum-safe keys and turning away old-style visitors. We recorded a real edge with Wireshark 4.2.2 in October 2026: first one quantum-safe visitor, then one old-style visitor.

What the recording shows

Frame	What happened	What Wireshark shows
4	The quantum-safe visitor says hello	Key share group 0x11ec (X25519MLKEM768), 1,216 bytes
6	The edge answers	Server Hello, group 0x11ec, 1,120 bytes
8 to 19	The web page travels	Only "Application Data", all scrambled
24	The old-style visitor says hello	Key share group x25519, 32 bytes
26	The edge refuses	Alert: Fatal, Handshake Failure

The edge's own log agrees: `X25519MLKEM768 TLS_AES_256_GCM_SHA384` for the first visitor, and `rejected … no suitable key share` for the second.

How to read it

- **The size gives it away.** An old-style key share is 32 bytes. The hybrid one is 1,216 bytes: 32 for X25519 plus 1,184 for ML-KEM-768. The server's reply is 1,120 bytes: 32 plus 1,088.
- **0x11ec is X25519MLKEM768.** Wireshark 4.2 shows it as "Unknown (4588)" because it is older than the name.
- **The page stays secret.** The page's text appears nowhere in the recording.
- **ID cards are hidden.** TLS 1.3 scrambles the certificates too, so you can't see ML-DSA in Wireshark. The console's Test connection shows the certificate's algorithm instead.

Trying it on Windows

1. Install Wireshark from wireshark.org. In the Npcap step, tick **Support loopback traffic**.

2. In PowerShell, make a CA and a certificate, then start a small web server: `acxelin-tls ca init --name "Demo Root CA" --dir pki --no-encrypt`, then `acxelin-tls ca issue server localhost --san 127.0.0.1 --dir pki --out web`, then `py -m http.server 8080 --bind 127.0.0.1`.
3. In a second window, start the edge: `acxelin-tls edge --listen 127.0.0.1:8443 --target 127.0.0.1:8080 --cert web\chain.pem --key web\key.pem`.
4. In Wireshark, record **Adapter for loopback traffic capture** with the filter `tls`.
5. Send a quantum-safe visitor: `acxelin-tls connect 127.0.0.1:8443 --ca pki\ca.crt --server-name localhost`.
6. Send an old-style visitor: `py -c "import ssl,socket; c=ssl.create_default_context(cafile='pki/ca.crt'); c.set_ecdh_curve('X25519'); c.wrap_socket(socket.create_connection(('127.0.0.1',8443)),server_hostname='localhost')"`. It fails with HANDSHAKE_FAILURE, which is the refusal you want to see.
7. Click each Client Hello and open **Extension: key_share** to compare the group and the size.

If you also record port 8080, you'll see the page in plain text on the short hop between the edge and the web server. That is why the service should run on the same machine as the edge, or on a private network.

Putting the edge in front of a public website

Yes, the edge can protect a customer's public website, using the `transition` policy with an ordinary public certificate as a backup.

Why a website needs a different setup

- Browsers already use the quantum-safe key exchange: current Chrome, Edge and Firefox use X25519MLKEM768.
- Browsers trust only public certificate authorities (Let's Encrypt, DigiCert and the like), never a company's own CA.
- As far as we know in October 2026, public CAs don't yet issue ML-DSA certificates for websites.
- The browser itself is getting ready. On our test laptop, Chromium 156 completed a handshake with an ML-DSA certificate from our CA, but only with certificate checks switched off. That shows the browser can handle the signature. It does not mean visitors' browsers will trust our CA.

So under `strict`, every ordinary visitor would be refused.

The setup

Setting	Value	Why
Policy	`transition`	Quantum-safe key exchange comes first, and older browsers still get in, so the site never breaks
Main certificate	ML-DSA, from our CA	For visitors that can check it
Backup certificate	An ordinary public one (Let's Encrypt, DigiCert)	What browsers actually check today
Target	The existing web server, moved to a local port	The website itself doesn't change

The backup certificate goes in the edge's settings file, not on the command line:

```
[[edge]]
name = "website"
mode = "terminate"
listen = "0.0.0.0:443"
target = "127.0.0.1:8080"          # their existing web server, moved to a
local port
policy = "transition"
cert = "pki/issued/www/chain.pem"   # ML-DSA certificate from our CA
key = "pki/issued/www/key.pem"
fallback_cert = "letsencrypt/fullchain.pem"   # public certificate browsers
trust
fallback_key = "letsencrypt/privkey.pem"
```

Start it with `acxelin-tls edge --config website.toml`. The edge refuses a backup certificate under any other policy or direction.

What it protects, and what it doesn't yet

	Status	Why
Recorded traffic (harvest now, decrypt later)	Protected now	Every modern browser uses the quantum-safe key exchange

	Status	Why
The site's identity (its certificate)	Still the old kind	A quantum computer could only fake it during a live connection, so recording traffic today doesn't help an attacker. A smaller, later risk.
Older browsers	The old kind, as today	Let in so the site keeps working. The console's Edges tab shows what share of visitors used the quantum-safe exchange.

When public CAs issue ML-DSA certificates that browsers trust, the customer swaps the backup for one of those. Nothing else changes.

The pitch: for a public website, "your visitors' traffic is safe from harvest now, decrypt later, today". For internal connections, partners and databases, use full `strict` with ML-DSA certificates and mutual TLS.

This follows the code: a backup certificate is accepted only on a `terminate` route with `transition`. It has not yet been run here with a real Let's Encrypt certificate.

The console

The console is a web page that runs on your own computer, and only your computer can open it. It does everything the `ca` commands do, without typing commands.

Opening it

1. Open PowerShell (Windows) or a terminal (Linux, macOS).
2. Run `acxelin-tls console`.
3. Your browser opens at `http://127.0.0.1:8900`, already signed in. Keep the terminal open, and press **Ctrl+C** there to stop.

Useful options: `--port 9000` (another port), `--home DIR` (where it keeps the CA and certificates; the default is `~/.acxelin-tls`), `--ca DIR` (manage an existing CA folder) and `--no-browser` (print the link instead). Each launch link works **once**. `acxelin-tls console link` prints a fresh one for a console that is already running. On a remote server, run `ssh -L 8900:127.0.0.1:8900 server`, then run `acxelin-tls console link` there and open the link on your laptop.

The tabs

Tab	What you do there
First run	Create the certificate authority: a name, and a passphrase typed twice.
Certificates	See the CA: its algorithm, how many certificates are valid, cancelled or running out within 30 days, and whether the cancellation list is current. Issue a certificate (server or client, name, extra names, days). The result shows where the files are and a ready-to-copy `acxelin-tls edge` command. Download, download with key, renew, cancel (two clicks), refresh the cancellation list, run maintenance, and see recent activity from the logbook.
Edges	Add a running edge by its metrics address to see its policy, its connections and which key exchanges were used. Test a connection to any host and port, with a chosen policy and, if you like, a client certificate for mutual TLS. It says "Connected, quantum-safe" or why not.
Health	Can this computer run Acxelin TLS (the OpenSSL version), and is the CA in good shape (cancellation list current, certificates not about to run out)?

How the console is kept safe

Protection	How it works
Only this computer	It listens on 127.0.0.1 only, and refuses any request that doesn't name this computer. That blocks a trick where an outside website poses as a local one (DNS rebinding).
A one-time launch link	The link carries a single-use ticket after `#`, a part browsers never send to servers or write in logs. The page swaps it for a session token kept in the tab's memory. There are no cookies, so another website can't ride on your session.

Protection	How it works
Guessing	Ten wrong tokens in a minute from one address lock that address out for the minute.
Planted scripts	The page runs only its own script, checked by its fingerprint. No outside scripts, fonts or frames.
Requests from other sites	A request from another website is refused.
The CA passphrase	Held in memory only while the console runs, never written to disk. A restart locks the CA again.
Secret keys	Reach the browser only when you choose *With key*, and never for a cancelled certificate or a locked key. Every download is written in the logbook.
Logbook	Every action goes to `audit.jsonl`, which only your user can read.

In the code: `acxelin_core/console.py` (the safety rules every Acxelin console shares), `acxelin_tls/console.py` (the 15 actions the page can ask for: status, create CA, unlock CA, certificates, issue, revoke, renew, maintain, download, refresh list, edges, add edge, remove edge, test edge, health) and `console.html` (the page).

Install, releases and the website

A customer downloads one folder, checks it, and runs one installer. No internet is needed after the download.

What a customer gets

Each release (now **1.0.2**, tag `tls-v1.0.2` on GitHub, on top of core 1.1.0) has two files:

- `acxelin-tls-1.0.2-wheelhouse.tar.gz`: the program and every library it needs, for Linux (Intel and ARM), Windows and Apple-silicon Macs, Python 3.11 to 3.14. Inside are the installers, the README, the manual, the licence, example settings and service files for Linux.
- `SHA256SUMS`: a fingerprint of each file, so the customer can prove the download wasn't damaged or changed.

There is also a Docker image, `ghcr.io/qubitman-hub/acxelin-tls:1.0.2`, which brings its own OpenSSL 3.5 for machines that don't have it.

Installing

1. Check the fingerprint (`sha256sum -c SHA256SUMS --ignore-missing` , or `Get-FileHash` on Windows).
2. Unpack it: `tar xzf acxelin-tls-1.0.2-wheelhouse.tar.gz` .
3. Run `sh install.sh` (Linux, macOS) or `install.ps1` (Windows).
4. The installer checks Python, finds OpenSSL 3.5, puts `acxelin-tls` where the system can find it, and runs `acxelin-tls doctor` .
5. `acxelin-tls try` shows the whole product in one minute: a CA, a plain web server, the edge in front of it, a quantum-safe visitor let in and an old-style visitor refused.

How a release is made

The owner pushes a tag such as `tls-v1.0.2` , always after the matching core tag. GitHub Actions then checks the version numbers match, runs every test, builds the folder for every platform, builds and scans the Docker image, writes `SHA256SUMS` , attaches a build certificate (proof that our own workflow built the file), and publishes the release.

The website

The website is plain HTML built from the repository and published on GitHub Pages whenever main changes.

- The source is `website/` , one folder per product. `tools/build_site.py` adds the shared header and footer to each page and writes `build/site` .
- The TLS section has six pages: **Overview**, **How it works**, **Set up**, **Manual** (built from `products/tls/docs/manual.md` , the same file the release ships), **Demo** and **Security**.
- The **Demo** page isn't made up. The build really runs `acxelin-tls try` and puts the real output on the page, so it always matches the code.
- Each product has its own section and menu, and its pages name only that product.
- Tests (`website/tests/test_site.py`) check the links, check that the pages work for screen readers and keyboards, and check that the pages say nothing the docs don't.

Use cases

Acxelin TLS has five groups of uses, and the code does all of them today. They are grouped by what the customer is protecting.

1. Services the customer already runs (the edge in `terminate` mode)

Use	Example	Setting	Tested here
Internal web apps and APIs	An HR portal, an internal API	`strict`	Yes, behind nginx
Databases	PostgreSQL, pgvector for AI search	`strict`, ready-made bundle	Yes, PostgreSQL
Message systems and caches	MQTT for sensors, Redis	`strict`, MQTT bundle	Yes, MQTT and Redis
Any other network service	A payment switch, a home-made protocol	`strict`	The edge doesn't care what the service speaks
Public websites	The customer's main site	`transition` + a public backup certificate	The code path only, not with a real Let's Encrypt certificate

2. Proving who is connecting (mutual TLS)

Use	Example
Partner connections	A bank lets only approved partners connect, each with its own ML-DSA certificate
Service to service	Internal services prove who they are to each other
Devices and branches	Each branch, laptop or sensor gets a certificate, cancelled in two clicks if lost

3. Systems that can't be changed (the edge in `originate` mode)

Use	How
Old client programs	An old program talks plain traffic to a local edge, which carries it quantum-safe to an edge elsewhere. The program never changes.

Use	How
Office-to-office links	Two edges, one at each end, turn an unprotected link between offices or data centres into a quantum-safe tunnel for that service

4. The customer's own certificate authority

Use	How
A quantum-safe internal CA	ML-DSA or SLH-DSA roots and middle CAs; issue, renew, cancel, cancellation list
Automatic sign-up for many machines	EST for machines with one-time codes; ACME for tools like certbot
Keys in hardware	AWS KMS or a hardware signing tool. Built, but not yet tried on real devices.

5. Rules and moving over

Use	How
US government rules	The `cnsa2` policy: ML-KEM-1024, ML-DSA-87 and AES-256 only
Moving over gradually	Start on `transition`, watch the Edges tab for who uses quantum-safe keys, then switch to `strict`
Proof for auditors	A logbook of every CA action, a log line per connection naming the key exchange, and Prometheus metrics
Demos and pilots	`acxelin-tls try` in one minute; bundles bring up nginx, PostgreSQL, pgvector or MQTT already protected

What it doesn't cover

- **Browser ID cards:** ML-DSA certificates for browsers wait for public CAs and browsers.
- **UDP traffic:** the edge handles TCP only, so no DNS or video calls.
- **Whole networks:** it protects one service at a time. Connecting whole networks or laptops is Acxelin VPN's job.

- **The hop to the app:** traffic between the edge and the application stays plain, so they should share a machine or a private network.

Code tour

Read it in this order: the bridge to OpenSSL, the TLS rules, the server, the edge, then the CA. The paths below sit under `core/src/acxelin_core/` and `products/tls/src/acxelin_tls/`.

Core (the shared machinery)

File	Lines	What to look at
`openssl.py`	521	The heart. Loads OpenSSL 3.5's TLS library from Python. `Context` sets the allowed key exchanges (`SSL_CTX_set1_groups_list`), signature algorithms and scrambling methods. `Connection.handshake()` runs the handshake with a deadline; `group()` says which key exchange was used; `export()` derives keys from the session (the VPN uses this).
`tls.py`	74	`Policy` (key exchanges, signatures, scrambling methods, the certificate algorithm required), `server_context()` and `client_context()` . Refuses to start if a certificate's key doesn't match the policy.
`server.py`	330	`Server` takes connections in threads, enforces the limits (`_admit` , `_take`), runs the handshake with a timeout, checks the client's serial against the cancellation list (`Revocation`), reloads certificates when they change (`reload_if_changed`), closes connections of newly cancelled clients (`close_revoked`) and keeps counts (`Stats`).
`pki/__init__.py`	493	The `CA` class: `init` , `issue` , `sign_csr` , `revoke` , `crl` , `renew` , `maintain` , `expiring` . Uses the ML-DSA support in `cryptography` .

File	Lines	What to look at
`pki/signers.py`	293	Where signatures come from: a key file, SLH-DSA through OpenSSL, AWS KMS or a hardware module's command. Builds the certificate with a stand-in key, then signs the final bytes again with the real signer.
`pki/est.py`	255	Machine sign-up (EST), server and client, with one-time codes stored only as fingerprints.
`console.py`	312	The safety rules behind every Acxelin console (see the console section).
`http.py`, `https.py`	171, 90	A small local web server for JSON, metrics and health, and just enough HTTPS over our TLS for EST.
`files.py`, `config.py`, `cli.py`	105, 63, 68	Private file writes (owner only), locking, TOML settings, prompts and logging.

The TLS product

File	Lines	What to look at
`policies.py`	20	The three policies, `strict`, `transition` and `cnsa2`. Short on purpose: this is the whole security decision.
`edge.py`	370	`Route` (one listening address to one target, with its policy, certificates, limits and timeouts) and the code that passes bytes between the TLS side and the plain side, in `terminate` or `originate` mode. Serves `/metrics` (Prometheus format) and `/status`.
`edge_commands.py`	169	The `edge`, `bundle`, `serve` and `connect` commands.
`ca_commands.py`	247	`ca init / issue / sign-csr / revoke / crl / maintain / list / token / publish / serve / enroll / acme / eab / csr`.

File	Lines	What to look at
`acme.py`	581	An ACME server (the Let's Encrypt method) in front of the CA: accounts, orders, http-01 challenges, and finishing with an ML-DSA request.
`bundles.py`	94	Makes a Docker Compose project with nginx, PostgreSQL, pgvector or MQTT already behind the edge.
`checks.py`	127	What `doctor` checks: the OpenSSL version, the CA folder and an edge's settings.
`console.py` + `console.html`	472 + the page	The console's actions and its single page.
`cli.py`	181	The `acxelin-tls` command; `try` runs a one-minute tour.

Design decisions worth discussing

- **Why talk to OpenSSL directly instead of using Python's `ssl`?** `ssl` can't set the TLS 1.3 key exchanges or say which one was used. Without that, you can neither enforce nor prove quantum-safe key exchange.
- **Why stop when in doubt?** An out-of-date cancellation list refuses everyone under mutual TLS. A visible outage is better than quietly letting a cancelled partner in.
- **Why only one library?** Fewer things to check and patch, and the product installs offline in a bank.
- **Tests that prove the security claims:** `test_downgrade.py` (old-style visitor refused), `test_secrets.py` (no passphrase or code is ever printed or stored), `test_security_regressions.py`, `test_scenarios.py` (real nginx, Redis, PostgreSQL, MQTT and a real Chromium browser), and the interop job (Node, curl and Go clients on OpenSSL 3.5). The browser test checks that Chromium gets the quantum-safe key exchange through `transition`, and that a client without ML-DSA is refused under `strict`.

Explaining it to a client

Start with how long their data must stay secret, show the one-minute demo, then answer the questions every client asks.

A five-step talk track

1. **The risk, in their words.** "Your partner connection carries account data that must stay secret for 10 years or more. Anyone recording that traffic today can read it once a quantum computer exists. Regulators such as the RBI, NIST and the EU now expect a plan to move over."
2. **The fix, without disruption.** "We put a guard in front of the connection. Your application doesn't change; only connections that use quantum-safe locks get through."
3. **Show it.** Run `acxelin-tls try` (one minute). The quantum-safe visitor gets through and the old-style visitor is refused.
4. **Show control.** Open the console: create the CA, issue a certificate for their partner, test the connection, cancel the certificate, test again: refused.
5. **The path.** Start with `transition` on one service, watch the Edges tab show which visitors already use quantum-safe keys, then switch to `strict`.

Questions clients ask

Question	Answer
Do we have to change our applications?	No. The edge sits in front, and the application keeps its plain local port.
Will our browsers and our customers' browsers work?	Yes. Browsers already use the quantum-safe key exchange. Public websites still need an ordinary public certificate, because browsers trust only public CAs, so use `transition` with a backup certificate there. Use `strict` for your own services and partners.
Is it standard, or your own crypto?	Standard: NIST FIPS 203 (ML-KEM) and FIPS 204 (ML-DSA), run by OpenSSL 3.5. We wrote no cryptography of our own.
Is it slow?	Only the opening handshake grows: about 2 KB more for the key exchange, and roughly 15 KB more for ML-DSA certificates and signatures (standard sizes from FIPS 203 and 204). After that the scrambling is the same AES-256-GCM as today. A speed job runs in our checks, but we haven't published timings yet; measure on the client's own hardware in the pilot.

Question	Answer
Does it phone home?	No. It contacts only the addresses you set.
What if your company disappears?	It runs offline from the files they hold, on standard OpenSSL, and the certificates are standard X.509.
Can we keep our CA key in our hardware module?	Yes, through KMS or any hardware signing command. This is built but not yet tested against a real hardware module, so offer a joint pilot.

Limits: what we don't claim

Say these before the client finds them. It builds more trust than any feature.

Limit	What it means in practice
Browsers don't trust ML-DSA certificates from your own CA, and public CAs don't issue them yet	Public websites still need an ordinary certificate (the `transition` policy). The key exchange is quantum-safe either way.
The CA machine is the crown jewel	Whoever holds `ca.key` can issue IDs. Keep the root switched off, use a middle CA, or use KMS or a hardware module.
KMS and hardware signing are not tested on real devices	Built and covered by automated tests; a pilot with the client's hardware module is the next step.
Cancelling isn't instant	A cancelled client works until its edge next reads the cancellation list: the next connection when the list is a local file, or within 60 seconds with `crl_url`.
The hop behind the edge is plain	Between the edge and the application, traffic isn't scrambled. Keep the application on the same machine (127.0.0.1) or a private network.
Platforms	The full test suite runs on Linux with OpenSSL 3.5. Windows and macOS run the tests in our checks, but are less used in real deployments.

Limit	What it means in practice
No published speed figures	A job in our checks measures the edge; figures for a client's hardware come from a pilot.
Not a certified product	It uses NIST-standard algorithms through OpenSSL, but has no FIPS 140-3 or Common Criteria certificate of its own.

Glossary

Term	In everyday words
ACME	The automatic way servers ask for and renew certificates (what Let's Encrypt uses). RFC 8555.
AES-256-GCM	The lock on the conversation after the handshake. Already strong enough against quantum computers at this size.
CA (certificate authority)	The passport office that signs ID cards for computers.
Certificate	A computer's ID card: its name, its public key and the CA's signature.
CNSA 2.0	The US National Security Agency's list of algorithms for secret systems; the `cnsa2` policy follows it.
CRL (cancellation list)	The list of cancelled certificates, signed by the CA and valid for 7 days.
CSR	A request for a certificate, made by a machine that keeps its own secret key.
Edge	Our guard that stands in front of a service and lets in only quantum-safe connections.
EST	A way for machines to sign up for and renew certificates by themselves. RFC 7030.
Handshake	The greeting at the start of a TLS connection, where keys are agreed and IDs checked.

Term	In everyday words
Harvest now, decrypt later	Recording scrambled traffic today to read it once a quantum computer exists.
HSM / KMS	Hardware, or a cloud service, that holds a secret key and signs with it, so the key never leaves.
Hybrid key exchange	Two locks on one door: the old X25519 plus the quantum-safe ML-KEM.
ML-DSA	Quantum-safe digital signature (FIPS 204). Sizes 44, 65 and 87: higher is stronger.
ML-KEM	Quantum-safe way to agree a secret key (FIPS 203). We use sizes 768 and 1024.
mTLS (mutual TLS)	Both sides show an ID card, not just the server.
OpenSSL 3.5	The widely used open-source TLS library; 3.5 is the first version with ML-KEM and ML-DSA.
Policy	Our rule for which algorithms a connection may use: `strict`, `transition` or `cnsa2`.
Revocation	Cancelling a certificate before it runs out, for example when a laptop is stolen.
SLH-DSA	A very cautious, hash-based quantum-safe signature (FIPS 205), used for roots that sign rarely.
TLS 1.3	The current version of the method that protects network connections.
X25519	Today's ordinary key exchange; a large quantum computer could break it.
X25519MLKEM768	The hybrid key exchange browsers and Acxelin TLS use by default.